Unit Testing

Introduction

- Software testing is the systematic process of evaluating software to determine whether it satisfies specified requirements and behaves correctly under expected and unexpected conditions.
- Testing helps
  - Detect defects early
  - Improve software reliability
  - Reduce maintenance costs
  - Enable safe refactoring
  - Increase developer confidence
  - Support long-term scalability
- Testing is not a debugging activity.
  - Debuggin identifies and fixes a known defect.
- Testing is proactive.
  - Intended to discover defects before deployment.

Unit Testing

- A software testing technique in which the smallest testable parts of an application (units) are verified independently.
- Unit is typically
  - A function
  - A method
  - A class
  - A module
  - A small isolated component.
- Main goal is to ensure each unit behaves correctly in **isolation**

Example:

```
function add(a, b) {
  return a + b;
}
```

Possible unit tests:

```
add(2, 3) === 5
add(-1, 1) === 0
add(0, 0) === 0
```

Why?

- Early defect detection
  - Bugs discovered during development are significantly cheaper to fix than bugs found in production.
- Refactoring Confidence
  - Bugs discovered during development are significantly cheaper to fix than bugs found in production.
- Documentation
  - Well-written tests explain *expected* behavior.
  - Communicates business rule directly.
- Better software design
  - Usually has lower coupling, clear responsibilities, better modularity

Good unit tests

- Should be
  - Fast
  - Isolated
    - Independent from external systems
  - Repeatable
    - Produces the same result every time
  - Automated
    - Runs without manual interaction
  - Clear
    - Easy to understand
  - Deterministic
    - No randomness
  - Focused
    - Tests one behavior only

FIRST

- Fast
- Independent
- Repeatable
- Self-Validating
  - Tests should automatically indicate pass/fail.
- Timely
  - Tests should be written close to production code development

Testing Types

Testing Type	Scope	Purpose
Unit Testing	Small isolated unit	Verify internal logic
Integration Testing	Multiple modules	Verify collaboration
System Testing	Entire system	Validate complete behavior
Acceptance Testing	User/business level	Validate requirements
End-to-End Testing	Real workflows	Simulate user actions

Anatomy

- AAA: Arrange Act Assert
- Arrange
  - Prepare inputs and dependencies
- Act
  - Execute the functionality
- Assert
  - Verify expected outcomes

```
// Arrange
const a = 2;
const b = 3;

// Act
const result = add(a, b);

// Assert
expect(result).toBe(5);
```

Assertions

- Assertions verify correctness

```
expect(result).toBe(5)
expect(user).toEqual(expectedUser)
expect(items.length).toBeGreaterThan(0)
expect(error).toThrow()
```

Test Case Design

- A test case defines
  - Inputs
  - Execution conditions
  - Expected outputs
- Good test cases cover
  - Normal inputs
  - Edge cases
  - Boundary conditions
  - Invalid inputs
  - Exceptional conditions

Boundary Value

- Many bugs occur at boundaries
- Example
  - 18 <= age <= 65
- You should test for
  - 17
  - 18
  - 19
  - 64
  - 65
  - 66

Positive & Negative Testing

- Positive testing
  - Verifies correct behavior for valid input
  - login(validUsername, validPassword)
- Negative testing
  - Verifies invalid input property
  - login(null, undefined)

Test Isolation

- A unit test should avoid external dependencies such as
  - Databases
  - file systems
  - APIs
  - Network services
  - Auth systems
- Because they introduce
  - slowness, complexity, non-determinism

Test doubles

- A test double replaces a real dependency during testing
- **Dummy**: Placeholder object
- **Stub**: Returns predefined data
- **Mock**: Verifies interactions
- **Spy**: Records calls
- **Fake**: Simplified implementation

Stubs

- Provides controlled responses

```
const fakeDatabase = {
  getUserById: () => ({ id: 1, name: "Alice" })
}
```

- Mocks
  - Verify interactions
  - Mocks are useful when behavior matters more than returned data
- Spies
  - Observe function calls
- Fakes
  - Provides a lightweight working implementation
  - An in-memory DB instead of MySQL

Code Coverage

- Code coverage measures how much code executes during tests.
- Common metrics
  - Line coverage
    - Executed lines
  - Branch coverage
    - Executed decision branches
  - Function coverage
    - Executed functions
  - Statement Coverage
    - Executed statements

- High coverage does NOT guarantee high quality.
- A single trivial test may produce 100% coverage while missing important cases
- Coverage is a *metric*, not a proof of correctness.

Test Smells

- A symptom of poor test design
- Common test smells:
  - Fragile Tests
    - Break after small code changes
  - Slow tests
    - Discourage execution
  - Duplicate tests
    - Repeated logic
  - Over-mocking
    - Excessive dependency simulation
  - Hidden dependencies
    - Difficult setup

Flaky Tests

- A flaky test passes sometimes and fails sometimes.
- Causes:
  - Race conditions
  - Timing issues
  - Shared state
  - Randomness
  - Network dependency
- Flaky tests reduce trust.

Deterministic testing

- A deterministic test always produces the same result.
- You need to avoid
  - Real time dependence
  - Random values
  - External APIs
  - Shared global state

DI: Dependency Injection

- Improves testability.
- Instead of
  - const database = new Database()
- Prefer
  - function UserService(database) {
    this.database = database
    }
- Easier mocking
- Better isolation
- More modular design

Pure functions

- Pure functions are highly testable
- Pure function
  - Produces same output for same input
  - Has no side effects
- Side effects
  - Writing files
  - Database access
  - network calls
  - Changing global variables
  - Console outputs (i/o)
- They complicate testing

TDD: Test Driven Development

- A development methodology based on short cycles.
- Cycle
  - Write failing test
  - Write minimal code
  - Make test pass
  - Refactor
- Benefits
  - Encourages modular design
  - Reduces unnecessary code
  - Improves maintainability
  - Creates comprehensive regression suite
- Criticism
  - Initial slowdown
  - Learning curve
  - Poorly designed tests can hinder refactoring
  - Over-testing trivial logic

Unit testing Strategies

- Black Box Testing
  - Tests based on I/O behavior.
  - Internal implementation is ignored.
- White Box Testing
  - Tests based on internal structure.
  - Knowledge of code internals is used.

CI

- Unit tests are usually integrated into CI/CD pipelines.
- Code commit → Build → Run unit test → Deploy
- Early failure detection
- Automatic validation
- Safer deployments

Best Practices

- Write small tests
  - Each test should verify one behavior
- Use clear names
- Keep Tests Independent
  - No shared mutable state
- Test behavior, not implementation

Unit Testing in Express

- A lot of tools
- Popular: **Jest**
  - Minimal configuration
  - Built-in mocking capabilities
  - Fast execution
  - ...

Jest

- In a single package:
  - A test runner
  - An assertion library
  - A mocking framework
  - A coverage tool
- Has built-in assertions
  - expect(result).toBe(5)
- Can run parallel test execution
- Can do coverage reporting

Supertest

- Used with Jest in Express
- Used for HTTP endpoint testing

```
request(app)
  .get("/users")
  .expect(200)
```

Demo

- Install Jest
  - nom install –save-dev jest
- Add to package.json
  - {“scripts”: {
    - “test”: “jest”
- npm test

- We have a math.js
- We create math.test.js

```
function add(a, b) {
  return a + b
}

module.exports = add
```

```
const add = require("./math")

test("adds two numbers", () => {
  expect(add(2, 3)).toBe(5)
})
```

Multiple test

```
const add = require("./math")

describe("add function", () => {

  test("adds positive numbers", () => {
    expect(add(2, 3)).toBe(5)
  })

  test("adds negative numbers", () => {
    expect(add(-2, -3)).toBe(-5)
  })

  test("adds zeros", () => {
    expect(add(0, 0)).toBe(0)
  })

})
```

Common matchers

- toBe
  - Primitive equality
- toEqual
  - Object/array equality
- toContain
  - see if a string is contained in another
- toBeTruthy, toBeFalsy
- toHaveLength
- etc.